

You're basically building **BeReal + TikTok for scrappy home cooks**, with an AI sous-chef living inside the pantry. Love it.

I'll walk you through **how to actually develop this new version** step-by-step:

- What the product is (in dev terms)
- App architecture (screens, navigation, data model)
- How each of the 3 pillars works:
 - Social media feed (friends + global + "BeReal for food")
 - Profile & badges
 - AI pantry / assistant (fridge vision, receipts, manual input)
- A phased build plan so you can implement this in Cursor without melting your brain.

Everything's grounded in your PDF spec: social-first, 35-and-under, resourcefulness, cost, and creativity.

1. Translate the PDF into a concrete product

From the doc, Cook'd's core:

- **Problem:** People buy new food instead of finishing what's in their fridge, wasting money and food.
- **Mission:**
 - Help young people **cook more often** (community + inspiration)
 - Help them **reduce waste & save money** (inventory + AI + cost focus)
- **Positioning:**
 - "BeReal for food" – spontaneous, authentic posts, not over-produced.
 - Social-media-first: home feed, discover page, reactions, DMs.

Three main feature pillars:

1. Social Media

- Friends feed (Home)
- Global / FYP (Discover)
- BeReal-style prompt (“post your real meal now”)
- Post card: photo of dish, mini blog, cost emphasis, ingredients used, star rating by creator, viewer reactions: “you cooked” / “you’re cooked”, save recipe, request recipe.

2. Profile

- Stats & badges (days cooked, average meal cost etc.)
- Pantry view
- Your posts
- Saved recipes

3. AI Assistant / Pantry

- Inventory of user’s food.
- Ways to input items:
 - Fridge vision (image → items)
 - Receipt scan (receipt → items)
 - Manual input
- AI recipes constrained to what they already have (“strictly these items”).

That’s the blueprint. Now: how do you actually build it?

2. Choose the stack (simple + mobile-first)

Given you're using Cursor and aiming for a **mobile app**, I'd go:

- **Frontend:**
 - **React Native + Expo** (easiest dev loop, good for iOS + Android).
 - Tailwind-style styling via **nativewind** (so you can reuse your Tailwind brain).
- **Backend (BaaS so you don't suffer):**
 - Supabase / Firebase / Lovable Cloud:
 - Auth (email, Apple, Google)
 - Postgres/Firestore for data
 - Storage for images
- **AI layer:**
 - One LLM / vision API that can:
 - Parse fridge/receipt images → item list
 - Generate recipes from inventory & constraints
 - Power the chat-style AI assistant
- **Real-time:**
 - Supabase channels / Firebase RTDB for live likes, reactions, DMs later.

If you prefer web/PWA, the rest of this still works; swap "React Native screen" for "React route".

3. Navigation & Screen Map

Bottom tab bar (from your doc):

1. **Home** – friends feed (following)

2. **Discover** – global/FYP
3. **Center “+” button** – BeReal-style post flow (camera → post composer)
4. **Chef** – pantry + AI assistant hub (inventory, fridge vision, receipt scan, recipe ideas)
5. **Profile** – user profile, stats, badges, pantry shortcut, settings

Core screens to define

- **Auth & Onboarding**
 - Login / Signup
 - Onboarding: basic profile, cooking goals, cost sensitivity (cheap vs fancy), diet.
- **Social**
 - HomeFeedScreen
 - DiscoverFeedScreen
 - PostCaptureScreen (camera)
 - PostComposerScreen (photo, mini blog, cost, ingredients)
 - PostDetailScreen (full recipe, reactions, comments)
 - RecipeRequestDMThreadScreen (creator ↔ viewer)
- **Pantry / AI**
 - PantryOverviewScreen
 - AddItemManualScreen
 - FridgeVisionCaptureScreen
 - ReceiptScanCaptureScreen
 - ReceiptReviewScreen (clean up OCR)

- AIChatScreen (assistant)
 - AIRecipeListScreen (recipes from current pantry)
 - **Profile**
 - ProfileScreen (stats, badges, posts grid)
 - SavedRecipesScreen
 - PantryShortcutScreen (maybe same data as pantry tab)
 - SettingsScreen
-

4. Data model (high-level)

You don't need full SQL now, but you *do* need mental tables:

Users

- id
- display_name
- username
- avatar_url
- bio
- created_at
- **Stats** (can be derived, but you may cache):
 - days_cooked
 - avg_meal_cost
 - meals_this_week

PantryItem

- id
- user_id
- name
- category (fridge/pantry/freezer)
- quantity
- unit (g, ml, pieces)
- expires_at
- source (manual, fridge_vision, receipt_scan, post_ingredients)

Post

- id
- user_id
- image_url
- caption
- cost_per_serving
- created_at
- visibility (friends, global)
- is_daily_drop (for BeReal-style “post now”)
- rating_by_creator (1–5)
- ingredients_used (link table to PantryItem or just names)
- recipe_id (optional: attached recipe)

Reaction

- id
- post_id
- user_id
- type (cooked, youre_cooked, like)

Recipe

- id
- creator_user_id (optional; AI recipes could have null)
- title
- steps
- ingredients (structured list)
- generated_by_ai (boolean)
- created_from_pantry_snapshot_id (optional)

PantrySnapshot (for AI history)

- id
- user_id
- items (JSON of what was present when recipe generated)

DM / RecipeRequest

- id
- from_user_id

- to_user_id
 - post_id
 - type (recipe_request, reply)
 - content
 - created_at
-

5. Building the 3 pillars

5.1 Social Media Feed (Home + Discover + BeReal)

Home Feed (friends)

Backend:

- Endpoint: GET /feed/home
 - Input: user_id, pagination cursor
 - Output: list of Post with User, Reaction counts, whether current user reacted.

Frontend:

- HomeFeedScreen:
 - Fetch with React Query.
 - Render list of PostCard components.
 - Pull-to-refresh, infinite scroll.

PostCard elements (UI inspired by your social inspo):

- Top row: avatar, name, time, cost badge (“\$2.40 / serving”)
- Photo card with rounded corners, drop shadow (use brand yellow/green as accent).

- Below photo:
 - Caption (1–2 lines, “mini blog”)
 - Ingredients pill row (“Used: leftover rice, frozen peas, egg”)
 - Creator’s rating (★ 4.5 by poster)
 - Reaction row:
 - “Cooked” button (I tried this)
 - “You’re cooked” (funny shock reaction)
 - Save icon (bookmark)
 - Comment count
 - Tap card → PostDetailScreen.

BeReal-style mechanic:

- Backend: store daily_drop_start_at per day.
- App: when user opens Home:
 - If they haven’t posted a daily meal in the current window, show a **banner**:

“Daily Cook’d: snap your real meal in the next 2 hours!”
 - CTA launches PostCaptureScreen directly.

Discover / FYP

Backend:

- Endpoint: GET /feed/discover
 - Mix of:
 - Top posts (reactions, saves)

- Cheap meals (low cost_per_serving)
- High waste-savings meals (if you track that later).

Frontend:

- DiscoverFeedScreen:
 - Same PostCard, but grid or vertical full-bleed cards like your inspo.
 - Filter chips at top: "\$5 or less", "Pantry clears", "Leftovers", "Vegan".
-

5.2 Profile & Badges

ProfileScreen layout (matching your branding inspo):

- Top: cover color block (brand teal), centered avatar in a white circle.
- Name, @username, small tagline.
- **Stats row:**
 - Days cooked
 - Avg cost/meal
 - Total meals
 - Approx \$ saved (if you estimate from AI or user input)
- Horizontal badges row:
 - "7-Day Streak", "Low-Cost Chef", "Fridge Raider", etc.
- Tabs:
 - **Meals** – grid/list of posts.
 - **Pantry** – quick view of pantry categories, CTA to Chef tab.

- **Saved** – recipes user saved from feed.

Badges logic:

- Compute in backend as scheduled job or on demand:
 - days_cooked – count of days with at least one post.
 - avg_meal_cost – avg of post.cost_per_serving.
- Decide thresholds:
 - 3 days in a row → “Getting Warm”
 - 7 days → “On a Roll”
 - 30 days → “Kitchen Deity” etc.

Frontend:

- BadgeCard components with playful icons & your brand colors.
 - Use these also on the **Profile > Badges** detail screen if you have a separate page.
-

5.3 AI Pantry & Assistant

This is the “Chef” tab.

PantryOverviewScreen

Sections:

- **Header card:**
 - “You have 18 items · 4 expiring soon”.
- **Category sections:**
 - Fridge, Freezer, Pantry.

- For each item: name, quantity, expiry badge (“3 days left”).

CTA row:

- “+ Add manually”
- “ Scan fridge”
- “ Scan receipt”
- “ Use what I have (AI recipes)”

Input methods

1. Manual input

- AddItemManualScreen:
 - Text input with autocomplete (common ingredients).
 - Category, quantity, unit, expiry date.
 - Save → call POST /pantry/items.

2. Fridge vision

- FridgeVisionCaptureScreen:
 - Camera view.
 - Take photo → upload to backend.
- Backend:
 - Vision model extracts likely items: “milk, eggs, tomatoes”.
 - Return list with confidence scores.
- FridgeVisionReviewScreen:
 - User confirms / edits items before committing to pantry.

3. Receipt scan

- ReceiptScanCaptureScreen: camera scanning receipt.
- Backend:
 - OCR → parse line items.
 - Map items to your ingredient vocabulary using an AI model.
- ReceiptReviewScreen:
 - Show each detected item, mapped ingredient, quantity.
 - User toggles which to add.
 - Confirm → add to pantry with source = 'receipt_scan'.

AI Assistant & recipes

AIChatScreen:

- Chat UI: user messages + assistant replies.
- Assistant context:
 - Current pantry (items).
 - User preferences (diet, budget).
- Example prompts:
 - “What can I cook under \$3 with what I have?”
 - “I only have 20 minutes.”

AIRecipeListScreen:

- When user taps “Use what I have”:
 - Send pantry snapshot + constraints to AI.

- Receive structured recipes (title, ingredients, steps, cost estimate).
- Show as a list of RecipeCards.
- From each recipe:
 - “Cook & Share” → prefill PostComposer with:
 - Photo placeholder empty
 - Ingredients list
 - Draft steps in caption.
 - “Save” → store in Recipe table.

Very important: by design, AI should **only use items in pantry** (and maybe essentials like salt/oil) to reinforce the “resourcefulness” angle in your problem statement.

6. Non-AI features tied into AI

This is where the app feels cohesive instead of like two separate apps.

- When user creates a **post**, the composer asks:
 - “Which pantry items did you use?” → multiselect from their inventory.
- When post is submitted:
 - Decrease quantities of those pantry items.
 - Use this to:
 - Track **days cooked**.
 - Track **waste avoided** over time.
- When a viewer **taps “Cooked”** on a post:
 - Offer to:

- Add missing ingredients to their **grocery list**, or
 - Generate a variation using their current pantry.
 - When a viewer **requests a recipe**:
 - Tap “Request recipe” → send DM to creator.
 - Creator can:
 - Type recipe manually.
 - Or tap “Let AI format my recipe nicely” – AI cleans their notes.
 - Once creator confirms, that recipe can:
 - Be attached to original post.
 - Be saved in both users’ recipe collections.
-

7. Phased build plan (how you actually implement this)

Phase 1 – Skeleton & Social MVP

1. Set up app

- Expo project + nativewind.
- Bottom tab: Home, Discover, +, Chef, Profile.

2. Auth

- Use Supabase/Firebase auth for sign in/up.

3. Basic social

- Post table.
- HomeFeedScreen, DiscoverFeedScreen with fake data, then wired to backend.

- PostCaptureScreen (camera) and PostComposerScreen (single photo, caption).
- Save cost_per_serving as a number even if user guesses.

4. Profile basics

- ProfileScreen showing name, avatar, simple stats from posts.
- Your posts grid.

You now have something people can actually use as a social food diary.

Phase 2 – Pantry & Manual AI

1. Pantry

- PantryItem table.
- PantryOverviewScreen, AddItemManualScreen.

2. AI recipes from pantry (no camera yet)

- Backend function that:
 - Reads user pantry items.
 - Calls AI model to generate recipes.
- AIRecipeListScreen & AIChatScreen.

3. Connect post → pantry

- On post composer, let users tag pantry items used.
- Decrement quantities.

Now the “waste reduction” story becomes real.

Phase 3 – Fridge Vision, Receipts & Recipe Requests

1. **Fridge vision**

- Implement camera → upload → vision model.
- Build review screen before adding to pantry.

2. **Receipt scan**

- Same pattern; reuse ReceiptReviewScreen.

3. **Recipe request DMs**

- Simple DM thread model between two users.
- “Request recipe” button on PostDetailScreen.
- Creator can respond with text or structured recipe.

Phase 4 – BeReal mechanic & polish

1. **Daily Cook’d**

- Backend field for daily window.
- Client logic to show “Post now” banner.

2. **FYP ranking**

- Basic algorithm: recency + reactions + low cost score bonus.

3. **Badges & advanced stats**

- Run a cron / scheduled function to compute badges.

4. **UI polish & branding**

- Apply your palette:
 - Background: soft beige

- Accent chips in mustard & sage
 - Primary CTAs in deep teal
 - Animations for reaction buttons, toasts for “Cooked!” etc.
-

8. What to actually do next in Cursor

Very concretely, your next steps in the codebase:

1. Define the new navigation structure

- Tabs + stack navigators for Social, Chef, Profile.

2. Refactor old components into this new mental model

- Keep reusable things like Button, PostCard, BadgeCard if they exist.
- Change them to match the new feature set (cost emphasis, cooked/you're cooked reactions).

3. Create basic types in types/:

- User, Post, PantryItem, Recipe, Reaction, Badge.

4. Wire HomeFeed + Profile to a real backend

- Even if you mock with Supabase local, make posts persist and show up on profile.

5. Add Pantry module with manual input first, then iterate to AI + camera.

Once that skeleton is in, every fancy feature (fridge vision, BeReal banner, badges, recipe requests) is “just” another screen + endpoint, not a re-architecture.